

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 untangle Namespace Reference	11
5.1.1 Detailed Description	11
6 Class Documentation	13
6.1 untangle::executor< taskT > Class Template Reference	13
6.1.1 Detailed Description	13
6.1.2 Constructor & Destructor Documentation	14
6.1.2.1 executor()	14
6.1.2.2 ~executor()	14
6.1.3 Member Function Documentation	15
6.1.3.1 add_task()	15
6.1.3.2 start()	15
6.1.3.3 stop()	16
6.1.4 Member Data Documentation	16
6.1.4.1 on_task_error	16
7 File Documentation	17
7.1 executor.hpp	17
Index	21

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);
    pool.start();

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); });
    }

    return 0; // ~executor finishes what was added, then stops the workers
}
```

the task type

The pool is a template on the task it runs, the way an execution is a template on its action. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The task type does not have to be a `std::function`. What an execution requires of it is a nested `result_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};
```

```
untangle::executor<my_task> pool(4);
```

A task type may take arguments, and `add_task()` binds them the way `execution::add_action()` does — at the door, where the caller still holds them:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();
pool.add_task([](int n) { return n * 2; }, 21);
```

What waits for a worker is one callable carrying its own arguments, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at `add_task()` and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking `int&` therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a `static_assert` in `add_task()`, rather than by an error from inside the binding.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on `_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; `start()` runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign `on_task_error`, before anything can throw.

`stop()` stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. `stop()` does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

`start()` after `stop()` restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

lifetime and failure

The destructor finishes what was added. It stops accepting, waits for the queue to empty and for every worker to go idle, and only then stops them. A task added before the pool goes out of scope has run by the time it does.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on stderr and dropped.

`add_task()` says whether the task was taken. It returns false before `start()`, after `stop()`, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool shutting down is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the shutdown from ever ending. One rule instead: a pool that is not running takes no work, whoever is asking. `add_task()` answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```
untangle::executor<std::function<void(void)>> pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};
```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_task()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on stderr instead, naming the worker.

A return value is still not collected: a continuous worker keeps none, so a task with something to report carries its own channel.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	11
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< taskT >	
Runs tasks on a fixed pool of untangle::async::execution workers	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	17
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers run in continuous mode behind a shared queue. A task goes straight to a free worker only when the queue is empty; otherwise it joins the back, and a worker takes the front as it frees up.

Chapter 6

Class Documentation

6.1 `untangle::executor< taskT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t` worker_count)
Builds worker_count workers. They do not run until `start()`.
- `~executor` ()
Finishes what has been added, then stops every worker.
- `template<typename... Args>`
`bool add_task` (`taskT` task, `Args &&... args`)
Queues a task bound to args, or hands it to a worker that has nothing to do.
- `std::size_t pending` () const
How many tasks are waiting for a worker.
- `void start` ()
Starts every worker, and starts the pool accepting work.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)>` `on_task_error`
Called with whatever a task threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename taskT>  
class untangle::executor< taskT >
```

Runs tasks on a fixed pool of `untangle::async::execution` workers.

Generated by Doxygen

Template Parameters

taskT	The type of the task the pool runs. One pool runs one signature - its arguments as much as its result - and a non-void return is run and dropped.
-------	---

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename taskT>
untangle::executor< taskT >::executor (
    std::size_t worker_count) [inline], [explicit]
```

Builds worker_count workers. They do not run until [start\(\)](#).

Parameters

worker_count	- How many workers to run. Must not be zero.
--------------	--

Exceptions

std::invalid_argument	- worker_count is zero. Such a pool would take work it could never run.
-----------------------	---

6.1.2.2 ~executor()

```
template<typename taskT>
untangle::executor< taskT >::~executor () [inline]
```

Finishes what has been added, then stops every worker.

Both waits are unbounded, and both report on stderr while they last, naming the workers they are waiting on.

Attention

A task still running here is refused if it adds more work, even though this is waiting for that very task. It is deliberate: a shutdown that took new work could be kept from ever ending by a task that re-adds itself. [add_task\(\)](#) answers false, so the task can see it.

6.1.3 Member Function Documentation

6.1.3.1 add_task()

```
template<typename taskT>
template<typename... Args>
bool untangle::executor< taskT >::add_task (
    taskT task,
    Args &&... args) [inline]
```

Queues a task bound to args, or hands it to a worker that has nothing to do.

The arguments are bound here, while the caller still holds them, and what waits for a worker is one callable carrying its own. That is the shape `async::execution::add_action()` offers at its door, and it is bound here for the same reason it is bound there: a queue holds callables, and a call site cannot be queued.

Remarks

The first free worker takes it, counting from the start, so a pool that is not busy keeps giving work to the same one. That is the intent: the worker that just ran a task is the warm one, and a free worker is free whichever it is.

Template Parameters

Args	- The argument types to bind to task.
------	---------------------------------------

Parameters

task	- The task to run.
args	- What to bind to it. Copied here and handed to the task as the pool's own lvalues when it runs, so a task taking a reference is given that copy rather than the caller's object, which may be gone by then.

Returns

true - the task was accepted. false - it was refused, by a pool that has not been started, has been stopped, or is being destroyed, or by a worker that has stopped. Either way it will not run. A task calling this from a worker thread is answered like any other caller.

6.1.3.2 start()

```
template<typename taskT>
void untangle::executor< taskT >::start () [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and refuses it again after `stop()` until this is called a second time - a stopped pool is restarted by it, workers and all. Calling it twice over is harmless.

6.1.3.3 stop()

```
template<typename taskT>
void untangle::executor< taskT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

Queued tasks stay unrun and a task added afterwards is refused. That holds whoever is asking, a task already running on a worker included - a pool that is not running takes no work. [start\(\)](#) gives the workers a new working life. Calling it twice is harmless.

Attention

It does not wait for the workers to leave their threads. Only the destructor does that, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 on_task_error

```
template<typename taskT>
std::function<void(std::exception_ptr)> untangle::executor< taskT >::on_task_error
```

Called with whatever a task threw. Assigned by the caller.

Nothing else reports a failed task: [add_task\(\)](#) has answered long before the task runs. Left unset, the pool warns on stderr instead.

Remarks

It receives a `std::exception_ptr`, because a task may throw what is not a `std::exception`. Rethrow it to read it.

Attention

It runs on a worker's thread, and several workers may be in it at once. Assign it before the first [add_task\(\)](#), and let nothing escape it.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00010 #pragma once
00011
00012 #include <algorithm>
00013 #include <async.hpp>
00014 #include <atomic>
00015 #include <chrono>
00016 #include <condition_variable>
00017 #include <cstdint>
00018 #include <deque>
00019 #include <exception>
00020 #include <functional>
00021 #include <memory>
00022 #include <mutex>
00023 #include <print>
00024 #include <stdexcept>
00025 #include <string>
00026 #include <thread>
00027 #include <type_traits>
00028 #include <utility>
00029 #include <vector>
00030
00031 namespace untangle {
00032
00039 template <typename taskT>
00040 class executor {
00041 public:
00050 explicit executor(std::size_t worker_count) {
00051     if (worker_count == 0) {
00052         throw std::invalid_argument("executor: worker_count must not be zero");
00053     }
00054
00055     workers_.reserve(worker_count);
00056
00057     for (std::size_t index = 0; index < worker_count; ++index) {
00058         auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00059
00060         // Tells the pool the moment this worker has room for more.
00061         next->on_finished = [this, index] { take_next_task(index); };
00062
00063         // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00064         next->on_error = [this, name = next->name](std::exception_ptr thrown) {
00065             report_task_error(name, thrown);
00066         };
00067
00068         // Adds the worker to the pool the destructor waits on.
00069         poll_.add(*next);
00070
00071         workers_.push_back(std::move(next));
00072     }
00073 }
00074
00086 ~executor() {
00087     // Whether the pool was still running when it was destroyed. A stopped one cannot empty its
00088     // queue - only a worker takes from it, through on_finished, and a stopped worker raises no more
00089     // - so its queue is not waited for.
```

```

00090     const bool was_running = running_.exchange(false);
00091
00092     {
00093         std::unique_lock<std::mutex> lock(mutex_);
00094
00095         // Waits for the queue to empty and every worker to go idle.
00096         auto interval = std::chrono::milliseconds(report_first_ms);
00097         auto waited = std::chrono::milliseconds(0);
00098
00099         while (!finished_cv_.wait_for(lock, interval, [this, was_running] {
00100             return nothing_running() && (pending_.empty() || !was_running);
00101         })) {
00102             waited += interval;
00103
00104             // Under the lock, so the counts agree with the predicate that just failed.
00105             std::println(
00106                 stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00107                 waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00108                 name_workers(&worker_execution::is_busy));
00109
00110             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00111         }
00112
00113         if (!pending_.empty()) {
00114             std::println(stderr,
00115                 "executor: {} queued task(s) dropped, the pool was stopped before they ran",
00116                 pending_.size());
00117         }
00118     }
00119
00120     stop();
00121
00122     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00123     // different question from the wait above: a worker can be idle and still be in its thread.
00124     auto interval = std::chrono::milliseconds(report_first_ms);
00125     auto waited = std::chrono::milliseconds(0);
00126
00127     while (poll_.is_running()) { // time of check
00128         const auto tick = std::chrono::milliseconds(poll_interval_ms);
00129         std::this_thread::sleep_for(tick);
00130         waited += tick;
00131
00132         if (waited >= interval) {
00133             std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00134                 waited.count() / 1000, count_workers(&worker_execution::is_running),
00135                 name_workers(&worker_execution::is_running));
00136
00137             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00138         }
00139     }
00140
00141     // time of use: no worker thread is left to reach workers_.
00142     workers_.clear();
00143 }
00144
00167 template <typename... Args>
00168 bool add_task(taskT task, Args&&... args) {
00169     // The call the pool will make, which is not quite the one written at the call site: what
00170     // reaches the task is the copies bound below, so that is the call that has to compile.
00171     static_assert(std::is_invocable_v<taskT, std::decay_t<Args>&...>,
00172         "executor: the task cannot be called with the arguments given to add_task()");
00173
00174     if (!running_) {
00175         return false;
00176     }
00177
00178     // Bound before the lock: it copies the caller's arguments, and nothing here needs the queue.
00179     task_call call = bind_task(std::move(task), std::forward<Args>(args)...);
00180
00181     std::lock_guard<std::mutex> lock(mutex_);
00182
00183     // The queue comes first: a task does not overtake one already waiting in it.
00184     if (pending_.empty()) {
00185         // Always from the start, so a free pool reuses its warmest worker rather than waking a cold
00186         // one. Under load the workers are busy and the queue below is what spreads the work.
00187         for (std::size_t index = 0; index < workers_.size(); ++index) {
00188             if (!workers_[index]->is_busy()) {
00189                 // A refused task is already destroyed, and stop() stops every worker together, so there
00190                 // is nothing to queue and no other worker to try.
00191                 return give_to_worker(index, std::move(call));
00192             }
00193         }
00194     }
00195
00196     pending_.push_back(std::move(call));
00197     return true;
00198 }

```

```

00199
00203 std::size_t pending() const {
00204     std::lock_guard<std::mutex> lock(mutex_);
00205     return pending_.size();
00206 }
00207
00215 void start() {
00216     // The workers first, so anything accepted after this is handed to one that is running.
00217     for (auto& one : workers_) {
00218         one->start();
00219     }
00220
00221     running_ = true;
00222 }
00223
00234 void stop() {
00235     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00236     // worker still runs; one arriving from here is refused.
00237     running_ = false;
00238
00239     // No lock: a worker waking from stop() runs on_finished, which takes mutex_.
00240     for (auto& one : workers_) {
00241         one->stop();
00242     }
00243 }
00244
00257 std::function<void(std::exception_ptr)> on_task_error;
00258
00259 private:
00266 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00267     if (on_task_error) {
00268         on_task_error(thrown);
00269         return;
00270     }
00271
00272     try {
00273         std::rethrow_exception(thrown);
00274     } catch (const std::exception& e) {
00275         std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00276     } catch (...) {
00277         std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00278     }
00279 }
00280
00289 struct task_call {
00290     using result_type = typename taskT::result_type;
00291
00292     result_type operator()() { return call(); }
00293
00294     std::function<result_type(void)> call;
00295 };
00296
00303 template <typename... Args>
00304 static task_call bind_task(taskT task, Args&&... args) {
00305     if constexpr (sizeof...(Args) == 0) {
00306         return task_call{std::move(task)};
00307     } else {
00308         // Captured by value and called as lvalues, which is all a call deferred to a worker can
00309         // promise: whatever was passed to add_task() may be gone by the time this runs.
00310         return task_call{[task = std::move(task), ... args = std::forward<Args>(args)]() mutable ->
00311             typename task_call::result_type { return task(args...); }};
00312     }
00313 }
00314
00315 using worker_execution = untangle::async::execution<task_call>;
00316
00323 [[nodiscard]] bool give_to_worker(std::size_t index, task_call task) {
00324     // add_action() takes the worker's action_mutex under mutex_. The two never cycle, because a
00325     // worker calls on_finished with its action_mutex released.
00326     return workers_[index]->add_action(std::move(task));
00327 }
00328
00332 void take_next_task(std::size_t index) {
00333     std::lock_guard<std::mutex> lock(mutex_);
00334
00335     if (!pending_.empty()) {
00336         task_call next = std::move(pending_.front());
00337         pending_.pop_front();
00338
00339         if (!give_to_worker(index, std::move(next))) {
00340             // The task is already destroyed, so it is reported rather than put back.
00341             std::println(stderr, "executor: {} refused a queued task, which is lost",
00342                 workers_[index]->name);
00343         }
00344
00345         return;
00346     }

```

```

00347
00348     if (nothing_running()) {
00349         finished_cv_.notify_all();
00350     }
00351 }
00352
00353 bool nothing_running() const {
00354     for (const auto& one : workers_) {
00355         if (one->is_busy()) {
00356             return false;
00357         }
00358     }
00359     return true;
00360 }
00361
00362 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00363     std::size_t count = 0;
00364
00365     for (const auto& one : workers_) {
00366         if ((one.get()->*state)()) {
00367             ++count;
00368         }
00369     }
00370     return count;
00371 }
00372
00373 std::string name_workers(bool (worker_execution::*state)() const) const {
00374     std::string names;
00375
00376     for (const auto& one : workers_) {
00377         if ((one.get()->*state)()) {
00378             names += names.empty() ? " : " : ", ";
00379             names += one->name;
00380         }
00381     }
00382     return names;
00383 }
00384
00385 static constexpr int poll_interval_ms = 50;
00386
00387 static constexpr int report_first_ms = 1000;
00388 static constexpr int report_max_ms = 30000;
00389
00390 mutable std::mutex mutex_;
00391 std::condition_variable finished_cv_;
00392
00393 async::execution_poll poll_;
00394
00395 std::deque<task_call> pending_;
00396 std::vector<std::shared_ptr<worker_execution>> workers_;
00397 std::atomic_bool running_ = {false};
00398 };
00399
00400 // namespace untangle

```


Index

- ~executor
 - [untangle::executor< taskT >, 14](#)
- add_task
 - [untangle::executor< taskT >, 15](#)
- executor, [1](#)
 - [untangle::executor< taskT >, 14](#)
- on_task_error
 - [untangle::executor< taskT >, 16](#)
- start
 - [untangle::executor< taskT >, 15](#)
- stop
 - [untangle::executor< taskT >, 15](#)
- untangle, [11](#)
- untangle::executor< taskT >, [13](#)
 - [~executor, 14](#)
 - [add_task, 15](#)
 - [executor, 14](#)
 - [on_task_error, 16](#)
 - [start, 15](#)
 - [stop, 15](#)